

Análise Léxica

Fabio Lubacheski

fabio.lubacheski@mackenzie.br

Análise Léxica

A **análise léxica** lida com técnicas para especificar e implementar um **analisador léxico**.

Um programa-fonte é informado para um **analisador léxico** sendo uma única cadeia continua de caracteres. O analisador léxico coleta caracteres e os agrupa logicamente de acordo com um **padrão**, atribuindo um **código interno** aos agrupamentos de acordo com sua estrutura.

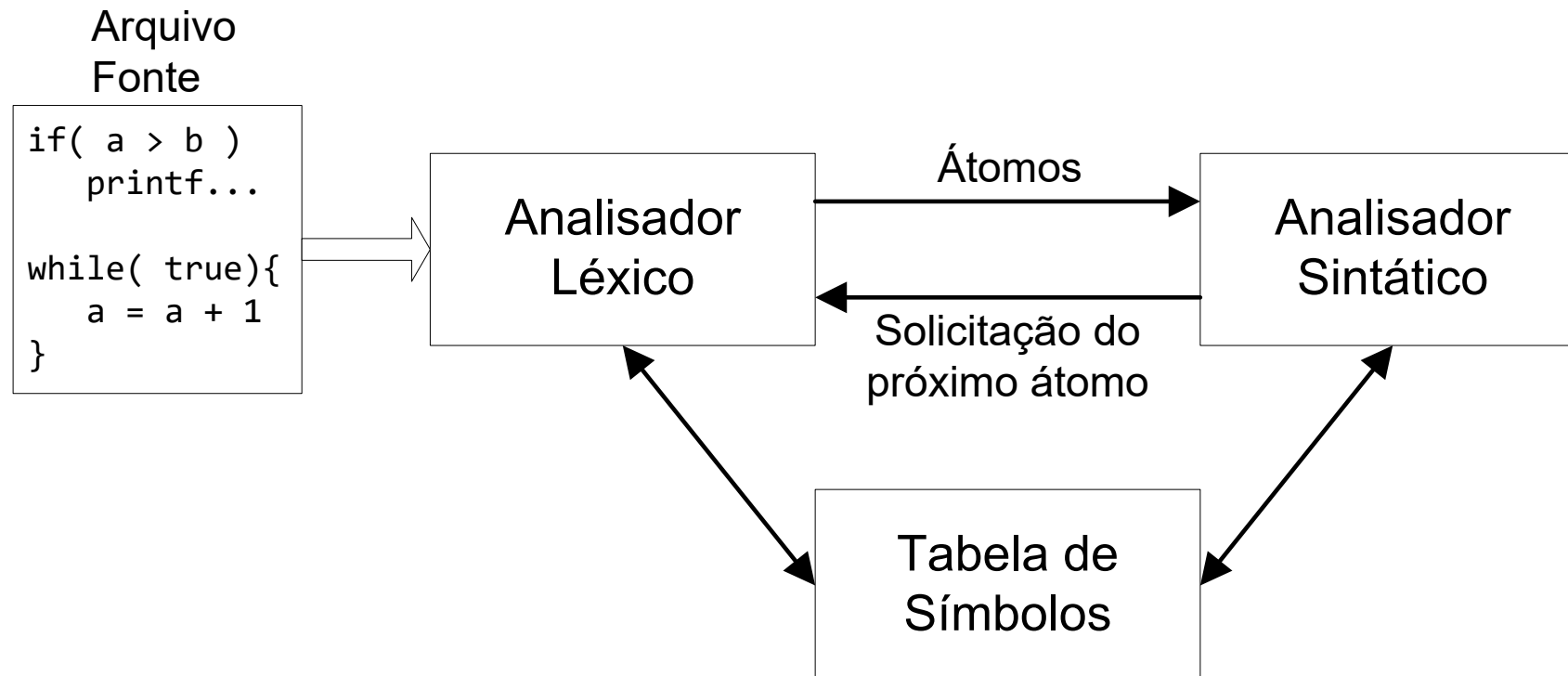
Os **agrupamentos lógicos** podem ser gerados por sequência de caracteres (**lexemas**) diferentes.

Os **códigos internos** desses agrupamentos são chamados de **tokens (átomos)**.

Analizador Léxico

O **analizador léxico** entende o programa fonte como uma **linguagem regular**.

Normalmente o **analizador léxico** é implementado como uma **subrotina do analizador sintático**.



Átomo

Na maioria das linguagens de programação, os seguintes **agrupamentos lógicos** são considerados **átomos**:

palavras-chaves (int, float...), **operadores**, **identificadores**, **constantes**, **cadeias de caracteres** (strings), e **símbolos de pontuação** como **vírgula**, **ponto e vírgula**, **dois pontos**, e **ponto**, etc.

Diferentes lexemas (sequência de caracteres) podem gerar um **mesmo token**, ou seja, um conjunto de **diferentes caracteres** podem determinar o mesmo **átomo**.

Atributos para átomos

Quando mais de uma sequência de caracteres (**lexema**) casa com um padrão para um átomo, o analisador léxico deve retornar informações adicionais sobre o átomo identificado.

Exemplo: **10** e **20** podem gerar o átomo **NUMERO**, mas é essencial para o gerador de código saber qual cadeia foi de fato reconhecida.

Como descrever a sequência de caracteres (**padrão**) que podem gerar um determinado **átomo** ?

Descrevendo o padrão dos átomos - expressão regular

Uma **expressão regular** é regra de formação para representar todas as combinações de caracteres que estão associadas a determinado **átomo**.

Para facilitar a escrita das **expressões regulares** dos átomos podemos nomear as expressões regulares (**definições regulares**), e assim a partir das definições regulares mais simples podemos definir outras definições mais complexas. Por exemplo:

DIGITO $\rightarrow 0|1|\dots|9$

Definição do representam o átomo **NUMERO**

NUMERO $\rightarrow \text{DIGITO}^+.\text{DIGITO}^+$

Como reconhecer os lexemas que geram o átomo NUMERO ?

Reconhecendo dos átomos – autômato finito

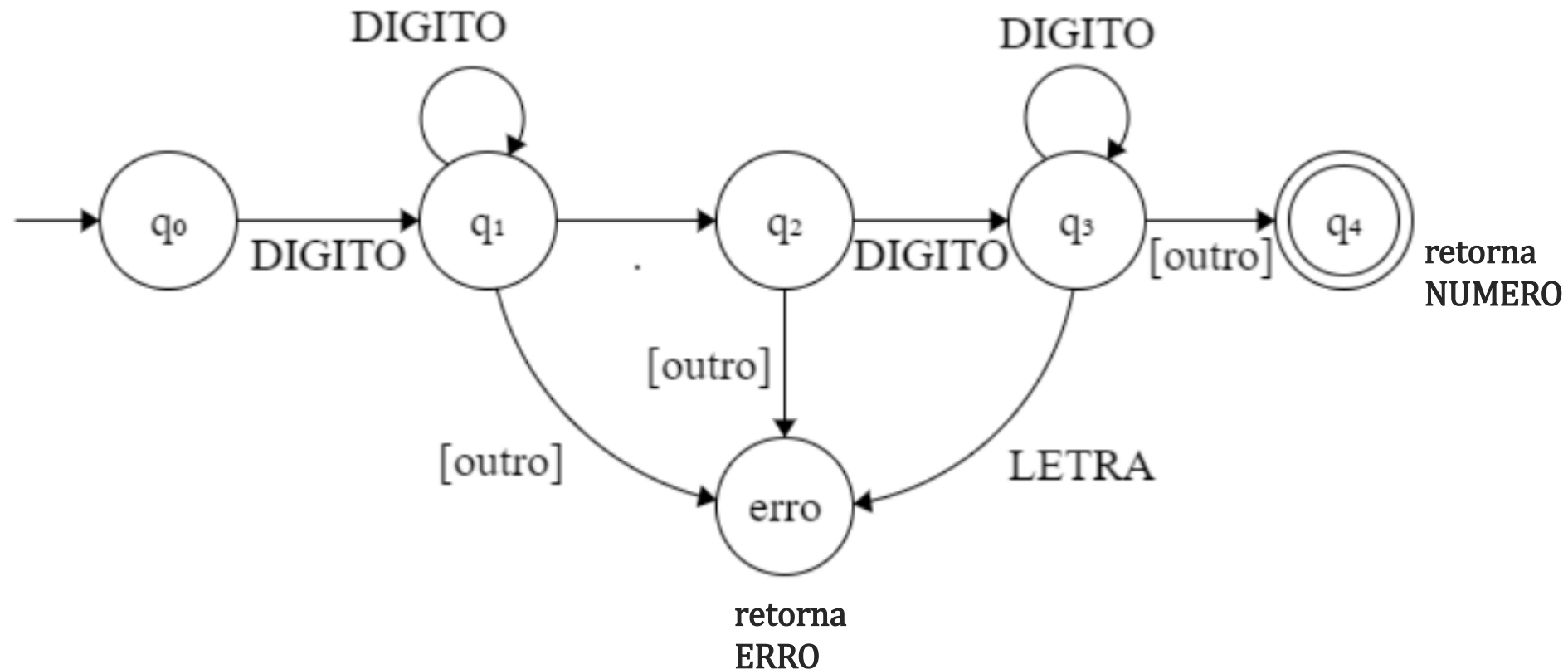
Um **autômato finito** aceitam cadeias de caracteres que casam com padrões definidos por uma **expressão regular**. Para nosso **analisador léxico** usaremos um **autômato finito** não tradicional.

Em autômato tradicional uma transição **sempre consome o símbolo de entrada**, além disso não há como especificar no autômato o que deve ser feito quando ocorre **erro no reconhecimento da cadeia** ou **quando a cadeia é aceita**, o que não reflete o comportamento das funções que reconhecem os átomos de uma linguagem de programação.

Reconhecendo o átomo **NUMERO**

Para a definição regular do átomo **NUMERO** temos o seguinte autômato.

NUMERO $\rightarrow$ **DIGITO**⁺.**DIGITO**⁺



Funcionamento do autômato

O caractere **[outro]** , com colchetes, é qualquer caractere diferente das transições em um estado do autômato, além disso o caractere não é consumido na transição

Um exemplo de como funciona caractere **[outro]** pode ser observado a partir do estado q_1 , note que o estado de **erro** é acessado quando após uma sequência de **DIGITO**, vier qualquer caractere que não seja um ponto (.), nesse caso o **analizador léxico** interrompe a execução e imprime uma **mensagem de erro**, por exemplo, **numero da linha # erro léxico**.

Caso o analisador chegue ao **estado final**, o analisador retorna a **constante numérica** do átomo **NUMERO** e o caractere **[outro]** não é consumido na transição.

Implementando a função para reconhecer átomo NUMERO

```
// código interno => atomo NUMERO
#define ERRO 0
#define NUMERO 1
// variavel global
char *buffer = "10.2;";
char lexema[20];

int reconhece_numero();

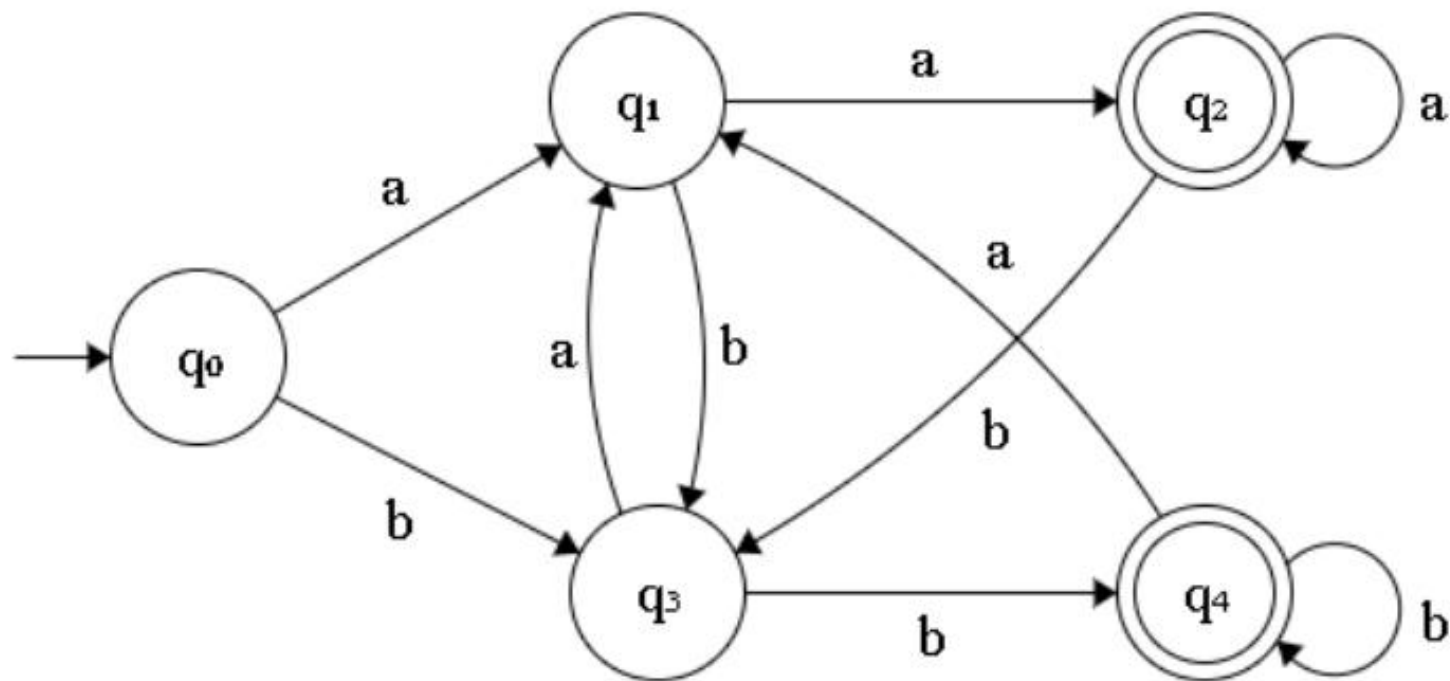
int main(){
    printf("Analizando: %s => ",buffer);

    if( reconhece_numero() == NUMERO )
        // e se quiséssemos o valor numérico ??
        printf("Atomo NUMERO reconhecido [%s].\n",lexema);
    else
        printf("Atomo NUMERO NAO reconhecido.\n");

    printf("fim de programa.\n");
    return 0;
}
```

Exercícios – Implementação máquina de estados usando goto

- 1) Baseado na implementação visto na aula EAD, agora você deve implementar uma função para o AFD abaixo que reconhece a linguagem $L = \{W \in \Sigma^* \mid \text{o sufixo de } W \text{ é } aa \text{ ou } bb\}$ onde $\Sigma = \{a, b\}$,



Exercícios – Revisão de Linguagens Formais e Autômatos

1) Seja o alfabeto $\Sigma = \{a, b\}$, desenvolva expressões regulares e autômatos para as linguagens a seguir.

$L1 = \{W \in \Sigma^* \mid W \text{ tem um número ímpar de } a's\}$

$L2 = \{W \in \Sigma^* \mid \text{o sufixo de } W \text{ é } aa \text{ ou } bb\}$

$L3 = \{W \in \Sigma^* \mid W \text{ não possui dois } a's \text{ consecutivos}\}$

2) Seja o alfabeto $\Sigma = \{a, b, c\}$, desenvolva uma expressão regular para

$L4 = \{a^i b^j c^k \mid i \geq 1 \text{ e } j \geq 2 \text{ e } k \geq 3\}$

$L5 = \{W \in \Sigma^* \mid W \text{ tem comprimento múltiplo de } 3\}$

$L6 = \{W \in \Sigma^* \mid W \text{ não inicia com } a\}$

$L7 = \{W \in \Sigma^* \mid W \text{ termina com } 3 \text{ } b \text{ consecutivos, e não mais que isso}\}$

Exercícios

3) Construa um autômato e, a partir do autômato, implemente uma função para reconhecer as palavras sobre o alfabeto $\Sigma = \{a, b, c\}$ para palavras geradas pela Expressão Regular $(a | \epsilon)(b | ca)^*c$. Qual é a linguagem gerada pela expressão regular ?

4) Considere a expressão regular (em notação estendida) para IDENTIFICADOR em uma linguagem de programação:

$$[A-Za-z][A-Za-z0-9]^*_([A-Za-z0-9]^+ | \epsilon)$$

Construa o autômato e, baseado no autômato, implemente uma função para reconhecer IDENTIFICADORES dessa linguagem de programação, onde:

$[A-Za-z]$ representa todas as letras do alfabeto; e

$[0-9]$ representa todos os dígitos de 0 até 9.

A sua função deve retornar uma constante representando o átomo ou 0 caso acontece erro no reconhecimento.

Exercícios

5) Considere a expressão regular para constantes numéricas de uma linguagem algorítmica com o seguinte formato abaixo:

$$(+|-)?n(,n)?(E(+|-)?n)?$$

Onde:

n é uma sequência de um ou mais dígitos, ou seja, $(0|1|2|3|4|5|6|7|8|9)^+$;

$x?$ significa que x é opcional, ou seja $(x|\epsilon)$.

Construa um autômato para a expressão regular acima, e baseado no autômato, implemente uma função que reconheça as constantes numéricas representadas pela expressão regular.

Exercícios – Revisão de linguagem C e arquivos

- 6) Escreva um programa na **linguagem C** que leia um arquivo texto e contabilize a quantidade ocorrência das letras **A..Za..z**, dígitos **0..9** e espaços em branco no arquivo texto. Ao final o seu programa deve imprimir uma listagem **quantidade da letras e dígitos em ordem crescente, quantidade de espaços em branco e o número de linhas no arquivo fonte**. Caso para determinado símbolo não tenha a ocorrência não é necessário imprimir a quantidade **0**, também **não é necessário** contar caracteres especiais com acento, como por exemplo **à** ou **ç**.
- 7) Escreva um programa na **linguagem C** que leia um arquivo texto e contabilize a quantidade de palavras nas linhas de um arquivo texto, a cada linha o seu programa deve informar a **quantidade de palavras encontradas**. Para esse exercício considere que as palavras serão formadas somente por letras maiúsculas e minúsculas sem caracteres especiais, como por exemplo **à** ou **ç**, considere também que as palavras são separadas por espaços em branco, vírgula, ponto e ponto e vírgula.

Exercícios – Revisão de linguagem C e arquivos

- 8) Reescreva as implementações, trocando as funções que utilizam goto para funções que utilizam while e switch .. case, ou seja, sem o uso de goto.
- 9) Escreva um programa que leia um arquivo fonte na linguagem C e converte todo o texto que está dentro de um comentário de uma linha (//) para maiúsculo. A saída do programa pode ser a tela do computador.
- 10) Escreva um programa em C que remova os comentários uma linha (//) de um programa fonte escrito na linguagem C padrão, considere que o arquivo está léxica e sintaticamente correto.

O programa fonte original deve ser lido de um arquivo, o seu programa terá como saída um novo arquivo fonte "limpo", sem os comentários (//) e tudo que estava dentro dos comentários, e é claro, o programa deverá continuar compilando sem erros como antes.

Exercícios - Implementação

11) Escreva um programa em C que remova os comentários várias linhas (`/* */`) de um programa fonte escrito na linguagem C, considere que o arquivo está léxica e sintaticamente correto.

O programa fonte original deve ser lido de um arquivo, o seu programa terá como saída um novo arquivo fonte "limpo", sem os comentários (`/* */`) e tudo que estava dentro dos comentários, e é claro, o programa deverá continuar compilando sem erros como antes.

Grato, alguma dúvida ?